Node.js + TypeScript 실습

AI 협업 개발 워크플로우 with Claude Code

코드 분석 · 구현 · 테스트 · 문서화까지, AI와 함께 일하는 방법

과정 안내

과정 목표와 결과물

훈련 목표

기존 개발 방식과 AI 기반 개발 방식의 차이를 설명할 수 있고 Claude Code로 코드 분석, 수정, 테스트, 문서화 흐름을 경험할 수 있다

설명	전통 개발과 AI 협업 개발의 차이
경험	분석 → 구현 → 테스트 → 문서화
확보	실습 저장소 + CLAUDE.md 템플릿

실습 결과물 — Focus Board (웹 화면 + API)

할 일·중요도·진행률을 관리하는 웹 화면 + REST API

```
GET    /api/tasks          # 목록 (?done=)
GET    /api/tasks/stats    # 진행률 %
GET    /api/tasks/:id      # 단건
POST   /api/tasks          # 생성
PATCH  /api/tasks/:id      # 수정·완료
DELETE /api/tasks/:id      # 삭제
```

TypeScript · Express 5 · Prisma 7 · SQLite · Vitest

결과물: 처음부터 직접 만든 focus-board-api · 테스트 전체 통과

과정 안내

모듈	주요 내용
1. AI 협업 개론	전통 vs AI 협업 · Vibe Coding · AI 코딩 도구 3분류
	Claude Code 동작 원리 · 설치 · 코드베이스 분석 실습
2. 요구사항·프롬프트	4단계 워크플로우 · 작업 분해(spec.md) · 프롬프트 패턴
	컨텍스트 관리 전략 · Plan 모드 계획 수립 실습
3. 프로젝트 구현	프로젝트 스캐폴딩 · Prisma 설정과 DB 스키마
	CRUD API 구현 · 웹 화면 구현 · Claude Code 문제 대처법
4. 검증·테스트	AI 코드 리스크 · 검증 체크리스트 · Vitest 테스트 · 보안
5. 문서화·운영	CLAUDE.md · README/API 문서 · .gitignore·환경변수

과정 안내

실습 준비물

- AI·머신러닝 배경지식은 필요 없습니다
- TypeScript는 **코드를 읽고 검토할 수준**이면 충분 — 작성은 Claude가 합니다
- 기본 터미널 조작(cd, ls, 명령 실행)은 할 수 있어야 합니다

Node.js 24 LTS

TypeScript 실습 런타임 · npm 포함

Claude Code

Pro 플랜 이상 유료 계정 필요

VS Code

Claude Code 확장 (선택)

Git · GitHub

변경 이력 관리 · diff 검토용 GitHub 계정 · gh CLI 로그인

설치 확인

```
$ node --version      # v24.x
$ claude --version    # 2.x
$ git --version
$ gh auth status      # GitHub 로그인
```



1. GitHub.com 선택
2. 프로토콜 HTTPS 선택
3. Git 자격 증명 연동 → Yes
4. "Login with a web browser" 선택 ·

과정 안내

실습 준비물 설치 방법

도구	Windows (PowerShell)	macOS (터미널)	설치 확인
Node.js 24	nodejs.org → **v24 LTS** 설치 파일	nodejs.org → **v24 LTS** 설치 파일	`node --version`
Claude Code	`irm https://claude.ai/install.ps1 | iex`	`curl -fsSL https://claude.ai/install.sh | bash`	`claude --version`
VS Code	`winget install Microsoft.VisualStudioCode`	`brew install --cask visual-studio-code`	확장 탭 → Claude Code
Git	`winget install Git.Git`	`xcode-select --install`	`git --version`
GitHub CLI	`winget install GitHub.cli`	`brew install gh`	`gh auth login`
GitHub 계정	github.com 에서 무료 가입	github.com 에서 무료 가입	`gh auth status`

Node.js는 **v24**로 설치하세요
10월 28일부터 'LTS'는 v26 · 실습 코드는 v24로 검증

Claude Code는 **공식 설치 스크립트** — 자동 업데이트
설치 후 **터미널을 새로 열기** · macOS brew 는 brew.sh

01

AI 협업 개론과 패러다임 전환

- 전통 개발 vs AI 협업 개발 차이
- Vibe Coding 개념과 적용 범위
- AI 코딩 도구 3분류 비교
- Claude Code 핵심 기능과 동작 원리 · 설치 · 코드베이스 분석 실습
- AI 코딩 효과적/주의 업무 구분

모듈 1

전통 개발 vs AI 협업 개발

단계	전통 개발	AI 협업 개발	사람의 역할
요구사항	회의·문서로 정리	자연어 요구사항 → AI와 spec.md 작성	의도 확정
설계	개발자가 구조 설계	Plan 모드로 AI가 설계안 제시	검토·승인
구현	한 줄씩 직접 작성	AI가 여러 파일 작성 + 명령 실행	범위·규칙 지정
테스트	마지막에 작성, 자주 생략	계획 단계에서 확정, AI가 실행·수정 반복	시나리오 검토
문서화	출시 직전 몰아서	코드와 함께 생성, CLAUDE.md로 규칙 공유	정확성 확인
온보딩	시니어에게 질문	코드베이스를 AI에게 질문	근거 확인

AI가 **작성**을 맡고, 사람의 일은 **판단과 검증**으로 이동합니다 — 생산성은 사람이 얼마나 잘 지시하고 확인하느냐에 달려 있습니다

모듈 1

바이브 코딩 vs 에이전틱 엔지니어링

바이브 코딩 (Vibe Coding)

- 아이디어만 던지고 결과를 받아쓰기
- 코드 검토 없이 수용
- 규칙·패턴은 AI 마음대로
- 막히면 원인 모른 채 재시도

적합 프로토타입·PoC · 일회성 스크립트 · 학습용 실험

에이전틱 엔지니어링 → 과정 목표

- 개발자가 전체 제어권 유지
- 개발 규칙·라이브러리·패턴을 명확히 제공
- 코드 리뷰와 테스트로 검증
- AI가 대부분 작성하되, 사람이 검토

필수 운영 서비스 · 인증·결제·개인정보 · 팀 공용 코드

버려도 되는 코드 → **오래 유지·공유되는 코드**

AI 시대 개발자의 역할 — 코드 작성·유지보수를 AI에 **전적으로 의존**하면, AI가 고칠 수 없는 문제가 생겼을 때 곤경에 처합니다

판단 질문 — **버려도 되는 코드인가?** · **다른 사람이 유지보수하는가?** · **실패하면 피해가 큰가?**

모듈 1

AI 코딩 도구 3분류 비교

강의 주제

① 코드 자동완성형

GitHub Copilot 인라인 제안, Tabnine

단위 한 줄 ~ 함수

방식 사람이 쓰고 AI가 제안

강점 흐름이 끊기지 않음

한계 프로젝트 전체 맥락이 약함

② AI 통합 IDE형

Cursor, Windsurf, VS Code 채팅·에이전트

단위 파일 ~ 기능

방식 에디터 안에서 대화·다중 파일 편집

강점 변경 사항을 눈으로 검토

한계 특정 에디터에 종속

③ 자율 에이전트형

Claude Code, Codex, Antigravity

단위 작업 ~ 프로젝트

방식 계획·실행·검증을 반복

강점 명령 실행·테스트·Git까지

한계 권한·검증 관리가 필요

AI의 자율성 ↑ 사람의 직접 개입 ↓

모듈 1

Claude Code란 — 핵심 기능과 동작 원리

저장소 안에 상주

프로젝트 폴더 안에서 직접 작업

코드베이스 전체 이해

구조·컨벤션·의존성을 스스로 읽고 파악

실행하는 에이전트

파일 편집, 명령 실행, 테스트, 커밋·PR까지 수행

외부 세계와 연결

MCP로 문서·브라우저·DB·GitHub 등과 통신



사용자는 이 루프의 **시작(지시)과 중간(승인)·끝(확인)**에 개입합니다

루프를 통제하는 3가지

컨텍스트 윈도우 — 무엇을 알고 일하는가 — 모듈 2

권한 모드 — 무엇을 승인 없이 해도 되는가 — 이번 섹션

CLAUDE.md — 어떤 규칙을 따르는가 — 모듈 5

모듈 1

Claude Code를 쓰는 3가지 방법

① 터미널 (CLI)	② 데스크톱 앱 (Code 탭)	③ IDE 확장 (VS Code)
설치 curl · PowerShell 설치 스크립트	**설치** 데스크톱 앱 설치 CLI 따로 설치 불필요	**설치** 확장 마켓플레이스에서 Claude Code 설치
특징 모든 명령·옵션 지원, `-p` 로 자동화	**특징** 여러 세션 병렬 실행 내장 터미널 · diff 뷰어	**특징** 열린 파일·선택 영역을 바로 컨텍스트로
추천 기본기 학습, 서버·SSH 작업	**추천** 여러 작업을 동시에 관리할 때	**추천** 에디터를 떠나지 않고 작업할 때

최근 흐름 — 데스크톱 앱이 **여러 에이전트 세션을 동시에 돌리는 작업대**로 재설계됨 (2026년 4월)

셋 모두 **같은 Claude Code**입니다 — 이 과정은 모든 기능이 드러나는 **터미널(CLI)** 로 배우고, 익숙해지면 데스크톱 앱에서 그대로 씁니다

모듈 1 · 실습

터미널(CLI) 첫 실행과 로그인

1

실행

프로젝트 폴더에서 `claude` 입력 → 대화형 세션 시작

2

로그인

`/login` → 계정 선택 → 브라우저에서 인증

3

확인·종료

`/status` 로 계정·모델 확인, `/exit` 로 종료

TERMINAL

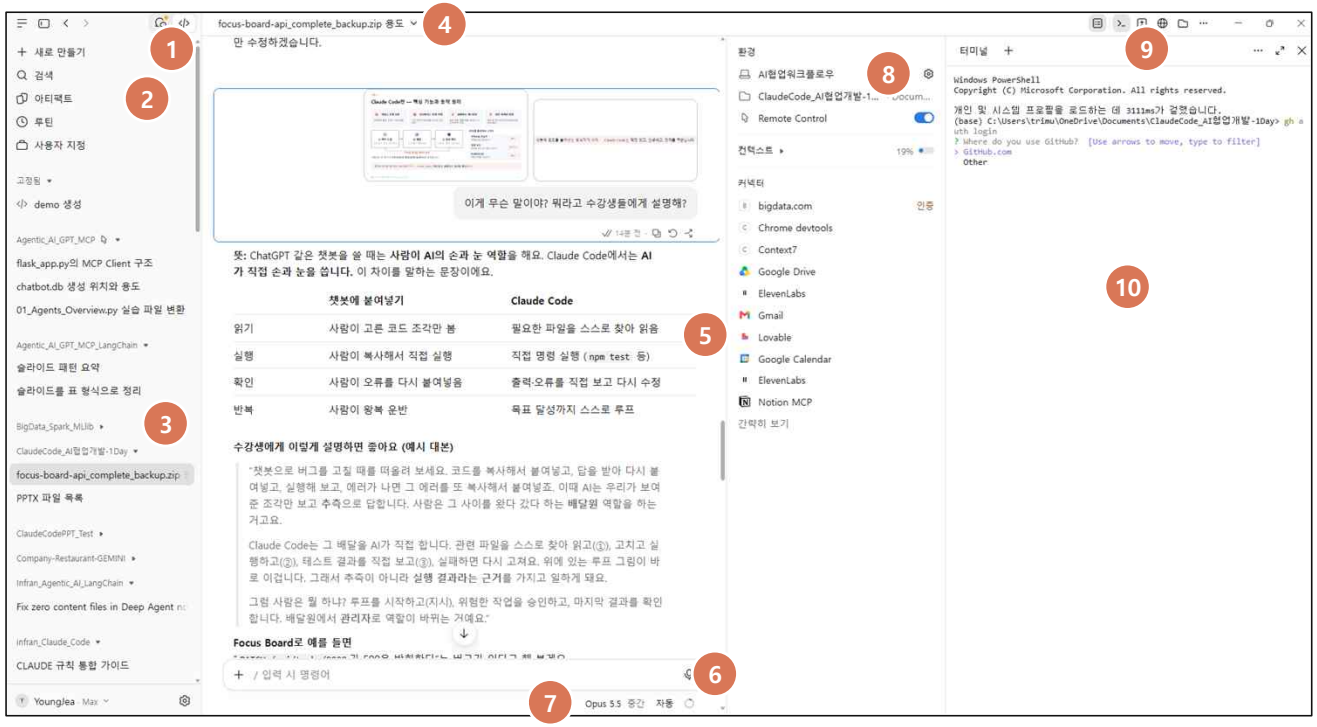
```
# 설치는 수업 전에 — 실습 준비물 설치 방법 참고
$ claude --version

$ cd my-project && claude
› /login
› /status
› /exit
```

`/logout` 계정 변경 · `claude -c` 마지막 세션 이어가기 · 공식 문서 **code.claude.com/docs**

모듈 1

Claude 데스크톱 앱 — 화면 구성



1 **Chat ↔ Code 전환**
상단 왼쪽 아이콘 — 일반 대화와 코딩 세션

2 **사이드바 메뉴**
새로 만들기 · 검색 · 아티팩트 · 루틴 · 사용자 지정

3 **세션 목록**
고정됨 + 프로젝트(폴더)별 그룹 · 세션 여러 개 병렬

4 **세션 제목**
지금 작업 중인 세션 · ▾ 로 세션 메뉴

5 **대화 영역**
지시 → 도구 실행 과정 → 결과를 한 흐름으로 표시

6 **입력창**
/ 명령 · @ 파일 참조 · + 첨부 · 음성 입력

7 **상태 표시**
모델(Opus) · 노력 수준 · 권한 모드(자동 등)

8 **세션 정보 패널**
작업 폴더 · Remote Control · 컨텍스트 % · 커넥터

9 **패널 열기**
세션 정보 · 터미널 · 브라우저 · 파일 패널 켜고 끄기

10 **내장 터미널**
사람이 직접 실행하는 명령 (예: gh auth login)

왼쪽은 **세션 고르기** · 가운데는 **Claude와 대화** · 오른쪽은 **지금 상태 확인** — 터미널(CLI)과 같은 Claude Code를 화면으로 다루는 것

모듈 1

Claude Code와 소통하는 3가지 방법

① CLI 명령어

세션 밖, 시작할 때

`claude`
대화형 세션 시작

`claude "버그 고쳐줘"`
작업을 지정하며 시작

`claude -p "질문"`
한 번 묻고 종료 (자동화)

`claude -c`
마지막 세션 이어가기

② 슬래시 명령어

세션 안에서

`/model`
모델 확인·변경

`/context`
컨텍스트 사용량

`/init`
CLAUDE.md 생성

`/rewind`
이전 상태로 되감기

③ 특수 기호

프롬프트 안에서

`@src/app.ts`
파일을 바로 참조

`!npm test`
셸 명령 직접 실행

`Shift+Tab`
권한 모드 전환

`ESC ESC`
되감기 메뉴

`@`로 파일을 지정하지 않으면 전체 검색으로 **시간과 토큰을 낭비**합니다 — Tab 자동완성과 함께 습관화하세요

모듈 1

권한 모드 — 무엇을 승인 없이 맡길 것인가

권한 모드	승인 없이 실행되는 작업	적합한 상황
자동 (Auto)	대부분 — 분류 모델이 위험 행동 차단	기본값 · 승인 피로 없이
수동 (Manual)	읽기만 — 변경·실행은 모두 승인 필요	처음 보는 코드, 민감한 작업
편집 자동 수락 (Accept Edits)	읽기 + 작업 폴더 안 파일 편집	변경 후 diff로 일괄 검토
계획 (Plan)	읽기 + 탐색 (수정 없음)	구현 전에 접근 방식을 합의

처음 보는 코드는 **수동** · 일반 개발은 **자동** · 큰 작업은 **계획 → 자동**

```
# CLI — Shift+Tab 으로 순환
manual → acceptEdits → plan → auto

# 시작할 때 지정
$ claude --permission-mode plan

# 데스크톱 앱 — 입력창의 모드 메뉴
자동 · 수동 · 편집 자동 수락 · 계획
```

격리 환경 전용

- `dontAsk` — 허용 목록만 실행 (CLI·CI/CD)
- `bypassPermissions` (권한 무시) — VM 전용

모듈 1

AI 코딩이 효과적인 업무 vs 주의할 업무

효과적 — 적극 위임	주의 — 사람이 결정, AI는 보조
• 보일러플레이트·프로젝트 스캐폴딩	• 보안·인증·결제·개인정보 로직
• CRUD·반복 패턴 코드	• 패키지 추가·버전 선택 (구버전 삽입 위험)
• 테스트 코드 작성과 실행	• DB 마이그레이션·데이터 삭제
• README·주석·API 문서	• 아키텍처·기술 스택 결정
• 낯선 코드베이스 설명·온보딩	• 요구사항 해석 — 의도는 사람만 앎
• 에러 원인 분석과 수정 제안	• 학습 시점 이후 바뀐 최신 라이브러리 API
• 테스트가 갖춰진 코드의 리팩터링	

'주의'는 금지가 아닙니다 — **결정은 사람이**, 작성은 AI가 돕되 **반드시 검증**한다는 뜻입니다

모듈 1

주요 슬래시(/) 명령 모음

묶음	명령	하는 일	나오는 곳
세션·계정	`/login` `/logout` `/status` `/usage` `/exit`	로그인·계정 전환 · 계정·모델 확인 · 플랜 사용량 · 종료	CLI 첫 실행과 로그인
컨텍스트	`/context` `/compact` `/clear`	사용량 확인 · 대화 요약 압축 · 대화 초기화	컨텍스트 관리 명령
작업 제어	`/model` `/rewind` (ESC ESC) · Shift+Tab	모델 변경 · 이전 상태로 되감기 · 권한 모드 전환	소통하는 3가지 방법 · 문제 대처법
프로젝트 규칙	`/init`	코드를 분석해 CLAUDE.md 생성·보완	CLAUDE.md 보완
확장·협업	`/plugin` `/commit-push-pr*` `/clean_gone*` `/install-github-app`	플러그인 설치 · 커밋→push→PR · 지운 브랜치 정리 · GitHub 자동 리뷰 연결	참고 — Git·PR 자동화
도움말	`/help`	전체 명령 목록과 설명	—

* commit-commands 플러그인을 설치하면 생기는 명령 · 터미널(CLI) 기준이며 데스크톱 앱은 일부 명령이 다르게 동작할 수 있음

입력창에 **/** 만 치면 목록과 설명이 뜹니다 — 외우지 말고 **찾아 쓰기**

02

요구사항 구조화와 프롬프트 설계

- AI 업무 워크플로우 4단계 모델
- 자연어 요구사항을 작업 단위로 분해 — spec.md 실습
- 효과적인 프롬프트 패턴
- 컨텍스트 관리 전략 · Plan 모드 계획 수립 실습

모듈 2

AI 업무 워크플로우 4단계 모델



테스트 계획은 ②**계획 단계에서 확정** — 구현 후에 설계하면 테스트가 코드에 끌려갑니다

모듈 2

자연어 요구사항을 작업 단위로 분해하기

자연어 요구사항

"할 일 관리 API 만들어 줘. 중요도도 있고, 완료 체크랑 진행률도 보여 줘."

사람이 제시

→

spec.md (무엇을)

- 기술 스택
- 데이터 모델 · 검증 규칙
- API 계약 (경로·상태 코드)
- 테스트 계획

AI 작성 · 사람 검토·확정

→

AI 분해 · 사람 검토·확정

작업 단위 (어떤 순서로)

1. 프로젝트 스캐폴딩
2. Prisma 스키마·마이그레이션
3. 생성·조회 API
4. 수정·삭제 API
5. 진행률 API · 웹 화면
6. 테스트 전체 통과
7. README·문서

분해 원칙

한 작업 = 한 세션

커밋 하나로 설명되는 크기

완료 기준이 있는 단위

테스트로 끝을 판정할 수 있게

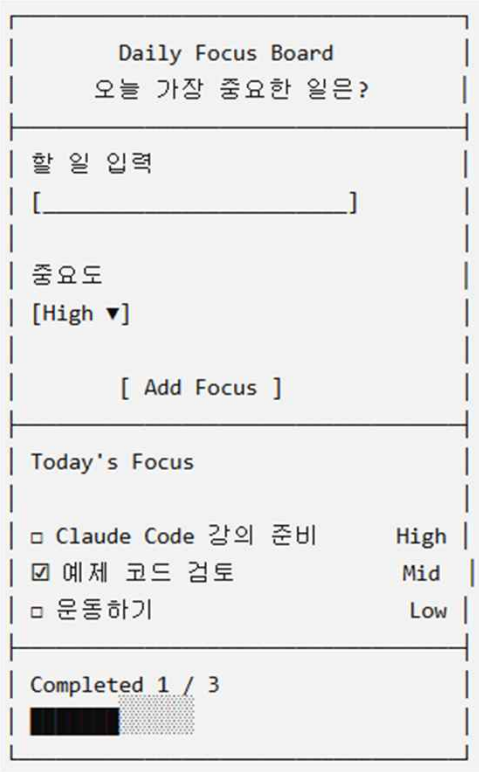
의존 순서대로

스키마 → API → 테스트 → 문서

모듈 2 · 실습

spec.md 작성하기 — 요구사항을 문서로 고정

ui-sketch.png



Ex 1) **요구사항이 막연할 때 — AI와 대화하며 구체화**

> @ui-sketch.png와 같은 할 일 관리 앱을 만들려고 해.
먼저 필요한 기능과 결정할 사항을 정리하고, 모호한 점은 나에게 질문해줘.
답변을 반영해 spec.md를 작성해줘. 코드는 아직 작성하지 마.

Ex 2) 요구사항이 구체적일 때 — 조건을 지정해 명세 작성

```
› 할 일(Task) 관리 웹 앱(화면 + REST API)을 만들려고 해.
  @ui-sketch.png 의 화면 구성을 참조해서
  아래 요구사항으로 my-spec.md 를 작성해 줘.
  - 기술: Node.js + TypeScript, Express 5,
          Prisma(SQLite), zod, Vitest
  - Task: title(1~100자), done,
          priority(HIGH/MEDIUM/LOW, 기본 MEDIUM)
  - 기능: 목록(완료 필터), 단건, 생성, 수정,
          삭제, 진행률(%)
  - 화면: 카드형 한 페이지(반응형), 진행률 바
  - API 계약은 메서드·경로·성공/실패 상태 코드 표로
  - 정상 / 경계값 / 실패로 나눈 테스트 계획 포함
  my-spec.md 만 만들고 코드는 아직 작성하지 마.
  모호한 점은 먼저 나에게 질문해 줘
```

모듈 2

프롬프트 엔지니어링 vs 컨텍스트 엔지니어링

구분	프롬프트 엔지니어링	컨텍스트 엔지니어링
질문	무엇을 어떻게 말할까	AI가 무엇을 알고 일하게 할까
핵심	지시문 작성	필요한 정보를 골라 주입
단위	한 번의 요청	세션 전체 · 프로젝트 전체
목표	더 정확한 응답	더 정확한 판단과 작업 수행

```
# 프롬프트 — 어떻게 말할까
이 버그의 원인을 분석하고,
수정 전에 계획을 먼저 제시해 줘.
```

```
# 컨텍스트 — 무엇을 알게 할까
CLAUDE.md · spec.md · 관련 소스 · 에러 로그
git diff · 공식 문서 · MCP 데이터
```

Claude Code에서는 **컨텍스트 쪽의 비중이 훨씬 큽니다** — 말을 잘 고르는 것보다 무엇을 읽게 할지가 결과를 가릅니다

모듈 2

효과적인 프롬프트 — 전제와 4가지 원칙

전제

생각하고, 계획하고, 작성하기

프롬프트를 쓰기 전에 작업부터 설계 — 후속 질문이 잦으면 계획 부족 신호 · Plan 모드로 먼저 합의

작업 설계

그 위에서 지키는 4가지 원칙

1. **간결하고 정확하게**
   핵심에 집중 — 중요하지 않은 세부 정보는 빼기
   프롬프트

2. **불필요한 컨텍스트 금지**
   '중요할 것 같은' 파일이 아니라 '실제로 중요한' 파일만 @ 참조
   컨텍스트

3. **AI를 '시험'하지 않기**
   함정을 안다면 숨기지 말고 해결책까지 프롬프트에 포함
   컨텍스트

4. **사용할 도구를 명시하기**
   웹 검색·MCP·서브 에이전트 — 알아서 쓰리라 기대하지 않기
   둘 다

모듈 2

프롬프트 패턴 — 실습에서 바로 쓰는 4가지

① 파일 참조

```
› @spec.md 의 API 계약대로
  POST /api/tasks 를 구현해 줘
```

② XML 태그로 자료 구분

```
› 아래 공식 문서를 근거로 설정해 줘
  <prisma-docs> …붙여넣은 문서… </prisma-docs>
```

③ 범위·제약 명시

```
› spec.md 만 수정하고
  애플리케이션 코드는 아직 바꾸지 마
```

④ 완료 기준 제시

```
› npm test 가 전체 통과할 때까지
  실패 원인을 분석해 수정해
```

네 가지를 한 프롬프트에 — 실전 예시

```
① @spec.md의 API 계약과 @prisma/schema.prisma를 참조해
   POST /api/tasks를 구현해 줘.

② <prisma-docs>
   [사용 중인 버전의 공식 문서 내용 붙여넣기]
   </prisma-docs>

③ 범위·제약:
   - 생성 API와 관련 테스트만 작성해.
   - 명세·데이터 모델·다른 기능은 변경하지 마.
   - 모호한 점은 먼저 질문해.

④ 완료 기준:
   - 정상·경계값·실패 테스트를 작성해.
   - 응답과 상태 코드가 명세에 맞고 npm test가 통과해야 해.
   - 변경 파일과 테스트 결과를 요약해.
```

정확한 레퍼런스를 주는 4가지 방법

문서 텍스트 직접 삽입 (가장 확실) | 공식 문서 URL 제공 | 웹 검색 지시 | Context7 MCP 사용

모듈 2

컨텍스트 윈도우 — Claude의 작업 기억

한 세션의 컨텍스트 윈도우에 함께 들어가는 것들



작업이 길어질수록 대화·명령 결과가 쌓여 남은 공간이 줄어듭니다 → 속도·정확도 저하 (Context Bloating)

토큰 — 처리의 기본 단위

- 영어 1토큰 ≈ 4글자, 한글 1토큰 ≈ 1~2글자
- 프롬프트·코드·답변 모두 토큰으로 계산
- 모델별 윈도우 크기·요금은 자주 바뀜 → 공식 문서 확인

관리 원칙

- **작업 단위마다 새 세션** — 하나의 작업에는 하나의 세션
- 필요한 파일만 @ 참조
- 오래 유지할 규칙은 대화가 아니라 **CLAUDE.md**에

모듈 2

컨텍스트 관리 명령 — /context · /compact · /clear

/context

사용량 확인

- 시스템·도구, 메모리 파일, 메시지, 남은 공간을 한눈에
- 자동 압축용 예약 공간도 표시
- `/usage` 는 플랜 한도 — 별개 개념

데스크톱 앱

화면의 사용량 표시로 바로 확인

/compact

대화 요약 압축

- 100% 근접 시 자동 실행
- 반복될수록 초기 지침 손실 위험
- 유지할 내용을 함께 지정해 수동 실행

데스크톱 앱

입력창에서 같은 명령 사용

/clear

대화 초기화

- 메시지 0%로 — 권한·MCP 연결은 유지
- 다른 문제를 시작할 때 사용
- 같은 회의실에서 화이트보드만 지우기

데스크톱 앱

사이드바 '새로 만들기' → 새 세션

```
› /compact 보안 규칙과 spec.md의 API 계약은 반드시 유지해 줘
```

모범 사례 — **하나의 작업에는 하나의 세션**, 지속할 규칙은 **CLAUDE.md**에 (압축·새 세션에도 항상 유지)

모듈 2

Plan 모드 — 평범한 프롬프트를 좋은 프롬프트로 업그레이드해 주는 장치

Plan 모드의 4가지 장점

1. **사전 정보 수집**
   관련 파일을 먼저 탐색
2. **명확성 확보**
   모호하면 AI가 되물어 프롬프트를 보완
3. **안전한 탐색**
   수정 권한 없이 구조 설계
4. **계획 저장**
   컨텍스트를 비워도 계획은 유지

계획이 끝나면 나오는 선택지

선택지	동작
Yes, and use auto mode (수락 및 자동 모드)	승인하고 Auto 모드로 구현
Yes, manually approve edits (수락 및 편집 허용)	승인하되 편집을 하나씩 검토
No, keep planning (거부)	계획 모드에 남아 수정 요청
데스크톱 앱 전용 (권한 수락 및 우회)	승인하고 권한 무시 모드로 구현 격리 VM에서만 사용

Shift+Tab “ Plan 모드 진입. Ctrl + G : 계획 직접 수정

모듈 2 · 실습

Plan 모드로 구현 계획 세우기

PROMPT

```
# Shift+Tab 으로 Plan 모드 전환 후
› @spec.md 를 기준으로 Focus Board 구현 계획을 세워 줘.
  - 작업을 의존 순서대로 나누고, 단계별로 변경할 파일을 적어 줘
  - 필요한 패키지 목록을 적어 줘 (설치는 계획 승인 후 진행)
  - 계획에 테스트 계획(정상/경계값/실패)과 통과 판정 명령(npm test)을 꼭 포함해 줘
```

좋은 계획의 4요소

- 구현 계획
- 변경할 파일
- 구현 순서
- 테스트 계획

승인 전에 사람이 확인할 것

- 스펙에 없는 기능이 끼어들지 않았나
- 작업 하나가 너무 크지 않은가
- 테스트가 실패 케이스까지 다루는가

이 계획이 모듈 3 실습의 작업 목록이 됩니다 — 계획 파일을 저장해 두세요

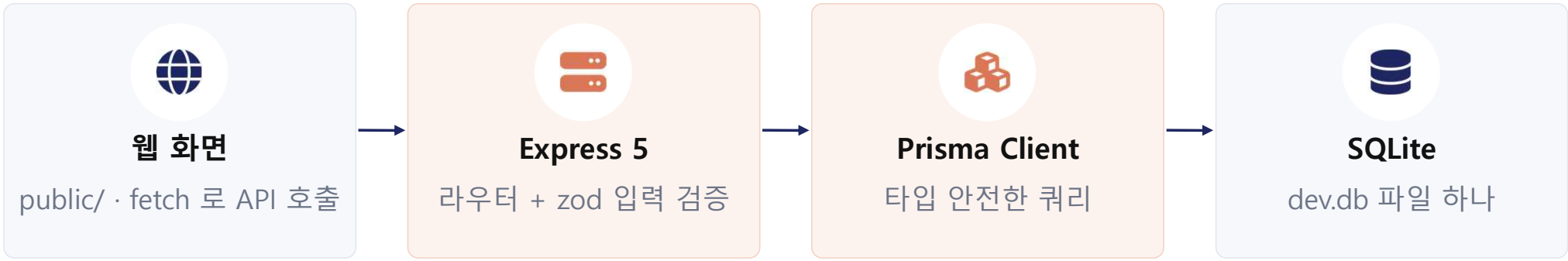
03

Claude Code로 프로젝트 구현

- Claude Code로 프로젝트 스캐폴딩
- Prisma 설정과 DB 스키마 생성
- CRUD API 구현
- 웹 화면 구현
- Claude Code 문제 대처법

모듈 3

실습 프로젝트 — Focus Board 구조



테스트: Vitest + supertest가 Express 앱을 직접 호출 (별도 test.db 사용)

```
focus-board-api/
├─ public/                 # 웹 화면 (HTML·CSS·JS)
├─ prisma/schema.prisma    # 데이터 모델
├─ prisma7.config.ts       # DB 주소 (DATABASE_URL)
├─ src/app.ts              # Express 앱 조립
├─ src/routes/tasks.ts     # CRUD 라우터
├─ src/schemas.ts          # zod 입력 검증
└─ tests/tasks.test.ts     # Vitest 테스트
```

왜 이 스택인가

- 요청 커리큘럼의 Prisma·CRUD 실습
- TypeScript 타입 = AI 출력 1차 검증
- 설치 없는 SQLite로 바로 실행

모듈 3 · 실습

프로젝트 스캐폴딩 — 설치는 AI가, 승인은 사람이

① 시작 · 사람

```
$ cd lab/focus-board-api
$ cp .env.example .env
$ git init
$ claude
```

② PROMPT · Plan 모드

```
› package.json, tsconfig.json 과 src/app.ts, src/server.ts 기본 구조를 만들어 줘.
  app.ts 는 createApp() 으로 분리해 테스트에서도 쓰게 하고 GET /health 만 먼저 구현해 줘.
  package.json 에 dev 스크립트를 넣고 포트는 3000 으로 해 줘.
  .gitignore(node_modules·.env·*.db·src/generated) 를 만들고 health 확인 후 첫 커밋해 줘.

  패키지 버전:
  - 설치 전에 npm view <패키지> dist-tags 로 최신 안정 버전을 확인해.
  - rc·beta·next 같은 프리릴리스는 쓰지 마.
  - prisma, @prisma/client, @prisma/adapter-better-sqlite3 는 같은 버전으로 맞춰.
  - 고른 버전을 계획에 표로 적고,  승인 후 그 버전으로 고정해서 설치해.

  완료 기준:
  - npm ls 에 peer 충돌 경고가 없고,
  - npx prisma -v 의 CLI·Client 버전이 같아야 해.
```

③ TERMINAL · 계획 승인 후 Claude가 설치 실행

계획 승인 창에서 패키지 이름과 버전을 확인하고 승인 — 설치는 자동

확인 — `npm run dev` 후 **localhost:3000/health** 가 `{"status":"ok"}` 반환

모듈 3

Prisma란 — TypeScript를 위한 ORM

SQL을 직접 쓰지 않고 TypeScript 코드로 DB를 다루게 해 주는 도구

SQL로 직접 쓰면

```
const rows = db.prepare(
  'SELECT * FROM "Task" WHERE "done" = 0'
).all();
```

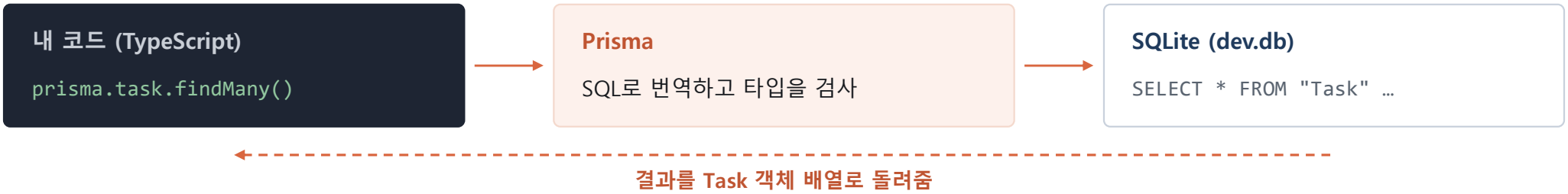
문자열이라 오타는 실행해 봐야 알고, 결과 타입도 모름

Prisma로 쓰면

```
const tasks = await prisma.task.findMany({
  where: { done: false },
});
```

TypeScript 코드라 자동 완성되고, 오타는 쓰는 순간 빨간 줄

Prisma가 하는 일 — 코드와 DB 사이의 통역사 (ORM: 객체 ↔ 테이블 연결)



모듈 3

Prisma와 DB 스키마의 관계

스키마 우선 (schema-first)
설계는 schema.prisma에서 — dev.db는 직접 고치지 않는다
순서 ① DB 위치 → ② 설계 → ③ migrate dev → ④ generate

2 **prisma/schema.prisma** 설계도

```
enum Priority { HIGH  MEDIUM  LOW }
model Task {
  id        Int      @id @default(autoincrement())
  title     String
  priority  Priority @default(MEDIUM)
  done      Boolean  @default(false)
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt
}
```

3 `npx prisma migrate dev`

dev.db SQLite 실제 테이블 — npx prisma studio 로 확인

TABLE "Task"	
id	INTEGER PRIMARY KEY AUTOINCREMENT
title	TEXT NOT NULL
priority	TEXT DEFAULT 'MEDIUM' ← enum
done	BOOLEAN DEFAULT false
createdAt	DATETIME DEFAULT CURRENT_TIMESTAMP
updatedAt	DATETIME NOT NULL

이력 → prisma/migrations/…_init/migration.sql

4 `npx prisma generate`

src/generated/prisma
타입이 붙은 쿼리 코드
(prisma.task 자동 완성)

드라이버 어댑터 (better-sqlite3)

src/app.ts

```
await prisma.task.findMany()
```

DB 주소 제공

1 **prisma7.config.ts**
DB 위치 = .env 의 DATABASE_URL
(file:./dev.db)

모듈 3 · 실습

Prisma 설정과 DB 스키마 생성

① 사전 준비 · 사람 (1회)

```
$ claude mcp add --scope user context7 \
    -- npx -y @upstash/context7-mcp
```

② PROMPT · Plan 모드

```
› Context7 MCP로 Prisma 7 공식 문서를 확인하고
  npx prisma init 으로 SQLite 설정을 시작해 줘.
  @spec.md 의 Task 모델로 schema.prisma 를 완성하고
  마이그레이션과 클라이언트 생성까지 해 줘.
```

왜 공식 문서부터?
Prisma 7은 설정 방식이 바뀌었습니다. 옛 자료로 배운 AI는
예전 방식으로 쓰기 쉬우니 최신 문서를 근거로 줍니다.

③ 계획 승인할 때 확인할 4가지 — 번호는 앞 장 관계도와 같음

1 **prisma7.config.ts** DB 위치

```
import "dotenv/config";   // .env 직접 읽기
datasource: { url: env("DATABASE_URL") }
```

2 **schema.prisma** 생성기 설정 (모델은 앞 장)

```
provider = "prisma-client"
output   = "../src/generated/prisma"
```

3 **migrate dev** DB와 이력 생성

```
$ npx prisma migrate dev --name init
```

4 **generate** 클라이언트 생성

```
$ npx prisma generate   # migrate가 대신 안 해 줌
```

확인 — `npx prisma studio` 를 열어 **Task 테이블**이 보이면 다음 단계로

모듈 3 · 실습

CRUD API 구현

메서드	경로	성공	실패
GET	/api/tasks	200	400 잘못된 필터
GET	/api/tasks/stats	200	-
GET	/api/tasks/:id	200	400 · 404
POST	/api/tasks	201 + Location	400
PATCH	/api/tasks/:id	200	400 · 404
DELETE	/api/tasks/:id	204	400 · 404

오류 형식 { error: { code, message, details? } }

PROMPT · 구체적으로 — Plan → Auto

```
› @spec.md 의 API 계약대로 tasks CRUD 를
  src/routes/tasks.ts 에 구현해 줘.
  (구현 조건)
  - 요청 본문과 id 는 src/schemas.ts 에
    zod 스키마를 만들어 검증
  - 없는 id 는 404, 검증 실패는 400
  - /stats 는 /:id 보다 먼저 등록
  - 테스트는 모듈 4 에서 작성하니 이번엔 생략
  - 완료 후 curl 로 각 API 상태 코드를 확인하고
    띄운 서버는 종료해 줘
```

PROMPT · 잘 모를 때 — Plan 모드

```
› @spec.md 의 API 계약대로 tasks CRUD 를
  구현해 줘(테스트는 모듈 4에서 작성).
  구현 전에 계획을 보여 주고, 내가 정해야 할
  것이나 주의할 점이 있으면 먼저 물어봐 줘.
  완료 후 각 API 를 직접 호출해 spec.md 의
  상태 코드와 맞는지 확인하고 서버는 종료해 줘.
```

모듈 3

생성된 코드 읽기 — 무엇을 확인할까

src/routes/tasks.ts (예시 — 이름·구조는 세션마다 다름)

```
router.post("/", async (req, res) => {
  const data = createTaskSchema.parse(req.body);        // ①
  const task = await prisma.task.create({ data });
  res.status(201)                                       // ②
     .location(`/api/tasks/${task.id}`)
     .json(task);
});

router.patch("/:id", async (req, res) => {
  const id = idParamSchema.parse(req.params.id);        // ①
  const data = updateTaskSchema.parse(req.body);        // ①
  const cur = await prisma.task.findUnique({ where: { id } });
  if (!cur) return sendNotFound(res, id);               // ③
  res.json(await prisma.task.update({ where: { id }, data }));
});
```

① 입력 검증

본문과 id 모두 `zod` 로 검증 — 실패는 400 VALIDATION_ERROR

② 상태 코드

생성 201 + Location, 삭제 204 — spec.md 4장 표와 한 줄씩 대조

③ 없는 리소스

`404` — 없는 id는 던지든 직접 응답하든, 조회·수정·삭제 모두

AI가 쓴 코드는 **사람마다 다릅니다** — 이름이 아니라 계약(상태 코드)·검증·오류 처리 세 곳을 확인합니다 · AI 교차 리뷰는 모듈 4에서

모듈 3 · 실습

웹 화면 붙이기 — 같은 API 위에 화면 한 장

PROMPT · Plan → Auto

```
› @spec.md 6장 화면 요구사항과 @ui-sketch.png 대로
  웹 화면을 만들어 줘.
  - public/ 에 index.html · style.css · app.js
    Express 가 정적 파일로 제공하게
    (app.ts 에 정적 파일 설정 추가는 괜찮아)
  - 데이터는 기존 /api/tasks API 만 사용
    (src/routes 의 API 코드는 수정하지 마)
  - 외부 라이브러리 없이, 카드형 · 반응형
  - 완료 후 /api/tasks 가 그대로 동작하는지 확인하고
    브라우저로 열 주소를 알려 줘
```

화면 구성 (spec.md 6장)

```
        Daily Focus Board
      오늘 가장 중요한 일은?
──────────────────────────────
할 일 [____________________]
중요도 [MEDIUM ▼]    [ 추가 ]
──────────────────────────────
□ Claude Code 강의 준비   HIGH
☑ 예제 코드 검토          MEDIUM
□ 운동하기                LOW
──────────────────────────────
완료 1 / 3  ■■■■□□□□   33%
```

화면도 **스펙에서 시작** — 그림 한 장이 긴 설명보다 정확합니다 · 라우트를 건드리지 않게 범위를 막아 **기존 API가 그대로 동작**하게

모듈 3 · 실습

실행과 기능 검증 — 브라우저에서 사람이 확인

TERMINAL

```
$ npm run dev
→ http://localhost:3000
```

기능 확인 체크리스트

✓ 할 일 추가 · 중요도 배지 · 빈 제목이면 오류 메시지
✓ 완료 체크 → 취소선 · 삭제 버튼
✓ 진행률 1/3 → 33% · 추가·완료·삭제 직후 바로 갱신
✓ 새로고침 · 서버 재시작 후 데이터 유지 (SQLite)
✓ 전체 완료 → 축하 메시지
✓ 모바일 폭 375px(DevTools 반응형 모드)에서 레이아웃 유지

문제 발견 시 — 직접 고치지 말고 다시 요청

```
› 할 일을 삭제해도 진행률 바가 바로
  갱신되지 않아. 원인을 찾아 수정해 줘.

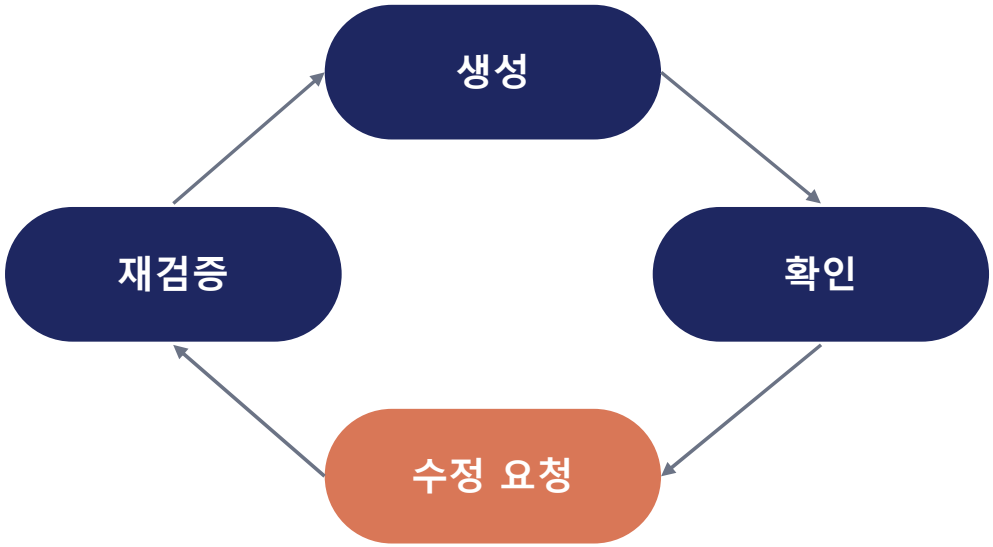
› 모바일 폭에서 추가 버튼이 너무 작아.
  사용성을 개선해 줘.
```

화면은 사람이 확인하는 이유

- 요구사항과 머릿속 그림이 맞는지는 의도를 아는 사람만 판단
- 레이아웃 깨짐은 코드만 봐서는 알기 어려움 — 직접 열어 확인

생성 결과는 매번 다릅니다 — 강사 화면과 같은지가 아니라 **요구사항이 구현됐는지**를 확인하세요

문제 대처법 ① — 직접 고치지 말고 다시 요청



좋은 수정 요청 = 현상 + 기대 + 근거

```
# 나쁜 예
› 안 돼. 고쳐줘.

# 좋은 예
› PATCH /api/tasks/9999 가 500 을 반환해.
  spec.md 상 404 여야 해. 서버 로그는
  <log> …붙여넣기… </log>
  원인을 먼저 분석하고 수정해 줘.
```

직접 고치면 AI는 그 이유를 모른 채 다음 작업에서 **같은 실수를 반복**합니다 — 반복되면 CLAUDE.md 규칙으로

모듈 3

문제 대처법 ② — 되감기(/rewind)와 증상별 처방

되감기 — 체크포인트와 /rewind

프롬프트를 보낼 때마다 코드 수정 직전 상태를 **자동 저장** — `ESC ESC` 또는 `/rewind` 로 선택해 복구

메뉴	동작
Restore code and conversation	코드 + 대화 모두 복구
Restore conversation	대화만 되감고 코드는 유지
Restore code	파일만 되돌리고 대화는 유지
Summarize from here	선택 지점 이후를 요약 압축

한계 — Bash 명령(rm, DB 변경)과 외부 편집기 수정은 추적되지 않습니다. 장기 기록은 **Git 커밋**으로.

증상별 처방

증상	흔한 원인	대처
엉뚱한 파일을 수정했다	요청을 다르게 이해함	/rewind 후 범위를 명시해 다시 요청
같은 실수를 반복한다	규칙이 대화에만 있음	CLAUDE.md에 규칙 추가
답이 점점 이상해진다	컨텍스트 비대화·압축 손실	유지할 내용 지정 /compact 또는 새 세션
없는 API·옵션을 쓴다	학습 시점 이후 바뀐 라이브러리	Context7·공식 문서를 XML로 제공
명령이 계속 차단된다	Auto 분류기·deny 규칙	의도 확인 후 직접 승인, 규칙 점검
테스트가 계속 실패한다	원인 추측으로 땜질	Plan 모드로 원인 분석부터 요청

체크포인트는 세션 안 즉각 복구, **Git**은 장기 기록 · 막히면 멈추고 **원인부터** — 같은 프롬프트 반복이 가장 비싼 대처입니다

04

AI 출력물 검증과 테스트 자동화

- AI 코드의 주요 한계와 리스크
- 검증 체크리스트 적용
- Claude Code로 유닛 테스트 생성·실행
- AI 코드 문제 패턴 식별 · 보안 기본 주의사항

모듈 4

AI 코드의 주요 한계와 리스크

비결정성

같은 요청에도 결과가 매번 다름

환각

없는 API·옵션·패키지를 그럴듯하게 사용

오래된 지식

최신 라이브러리를 구버전 방식으로 작성

맥락 누락

프로젝트 규칙·기존 패턴을 무시하고 작성

과잉 구현

요청하지 않은 기능·추상화로 복잡도 증가

그럴듯한 오류

실행은 되지만 경계값·예외 처리가 틀림

대응의 공통점 — **사람이 기준(스펙·테스트)을 먼저 정하고**, AI 출력은 그 기준으로 검증합니다

모듈 4

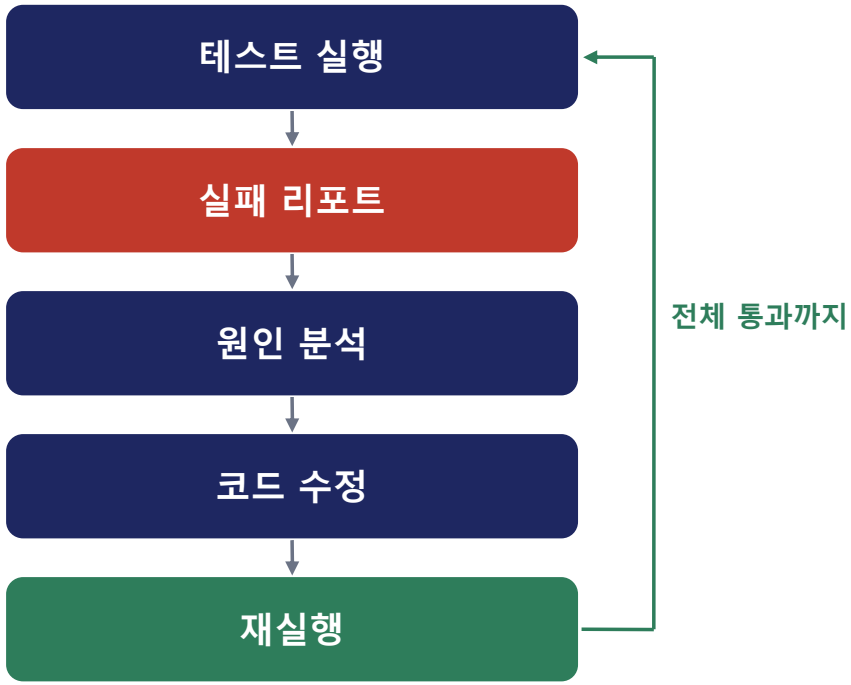
검증 체크리스트 — 누가, 어떻게 확인하나

체크 항목	담당	확인 방법
요구사항대로 동작하는가	사람	spec.md API 계약과 실제 응답 대조
상태 코드·오류 형식이 맞는가	AI + 사람	테스트로 자동 확인, 결과는 사람이 확인
입력 검증이 빠짐없는가 (빈 값·길이·타입)	AI → 사람	새 세션에서 "입력 검증 누락 검토해 줘"
테스트가 의미 있는 시나리오인가	사람	경계값·실패 케이스가 있는지 직접 확인
비밀정보·불필요한 파일이 섞이지 않았나	사람	`git diff` · `git status` 로 변경 범위 확인
불필요하게 복잡한 코드는 없는가	AI	새 세션에서 단순화 관점 리뷰 요청

리뷰는 **새 세션**에서 — 코드를 작성한 세션은 자기 결정을 정당화하는 쪽으로 기울기 쉽습니다

모듈 4

AI에게 검증 수단을 — 테스트 피드백 루프



왜 AI에게 테스트가 필요한가

- 실패 리포트의 **기대값 vs 실제값**이 수정 방향을 알려 줌 — 추측 대신 근거
- "전체 통과"라는 **명확한 완료 기준**이 생김
- 다음 수정이 기존 기능을 깨뜨리면 즉시 발견

```
## 완료 기준  (CLAUDE.md)
- 테스트 계획의 테스트를 모두 작성하고
  npm test 가 전체 통과해야 완료로 간주한다
- 실패 시 원인을 분석해 수정하고 재실행한다
```

통과 판정 명령을 적어 두면 Claude가 스스로 실행해 확인합니다

모듈 4

Vitest 기초 — describe · it · expect

tests/tasks.test.ts

```
import request from "supertest";
import { describe, expect, it } from "vitest";

describe("POST /api/tasks", () => {
  it("priority 를 생략하면 MEDIUM 이다", async () => {
    const res = await request(app)
      .post("/api/tasks")
      .send({ title: "기본 중요도" });

    expect(res.status).toBe(201);
    expect(res.body.priority).toBe("MEDIUM");
  });
});
```

함수 + 기대값 비교 = 테스트

describe	관련 테스트 묶음
it	테스트 하나 = 확인할 동작 하나
expect	기대값 비교 — 다르면 실패 리포트
supertest	서버를 띄우지 않고 Express 앱 호출

```
$ npx vitest run            # 전체 1회
$ npx vitest run -t "MEDIUM"    # 이름으로 지정
$ npx vitest                # 감시 모드
```

모듈 4 · 실습

Claude Code로 유닛 테스트 생성·실행

PROMPT

```
› @spec.md 의 테스트 계획대로 tests/tasks.test.ts 를 작성해 줘.
  - 정상 / 경계값 / 실패 케이스를 describe 로 구분
  - 별도 test.db 를 사용하고 dev.db 는 건드리지 마
  - npm test 가 전체 통과할 때까지 원인을 분석해 수정해
```

1 spec.md

사람 · 배포 파일

5장 테스트 계획에 파일 경로와 정상·경계값·실패 케이스를 정의

→

2 Plan 모드

Claude · 사람 승인

Plan 모드 실습 구현 계획에 테스트 계획 포함

→

3 테스트 작성

Claude

위 프롬프트로 테스트 파일 작성 + test.db 준비

→

4 npm test

Claude · 반복

실패 → 원인 분석 → 수정 → 재실행 전체 통과까지

추가 파일 · 실행 결과 예시 (테스트 수는 다름)

```
vitest.config.ts       # test.db
tests/global-setup.ts # migrate
tests/tasks.test.ts    # 테스트

$ npm test
 ✓ tests/tasks.test.ts (22 tests)
 Test Files  1 passed (1)
      Tests  22 passed (22)
```

AI가 만든 테스트도 **검토 대상** — 실패 케이스가 충분한지 사람이 확인

모듈 4

AI 코드 문제 패턴 식별하기

패턴	이런 신호가 보이면	대응
통과하기 쉬운 테스트	구현에 맞춘 기대값, 실패 케이스 없음	테스트 계획을 먼저 확정 (TDD)
과도한 모킹	DB·핵심 로직까지 가짜로 대체	외부 I/O만 모킹, DB는 실제 test.db 사용
구버전·가짜 종속성	package.json 버전이 AI 기억 속 옛 버전, 존재 않는 패키지나 버전	npm install 로 설치, 실행 전 사람 승인
검증 누락	req.body 를 그대로 저장(클라이언트가 보낸 JSON을 검사 없이 그대로 DB에 삽입)	zod 검증을 CLAUDE.md 규칙으로
오류 삼키기	빈 catch, 무엇이든 200 응답	오류 상태 코드를 테스트로 고정
요청 범위 초과	요청하지 않은 파일까지 변경	`git diff` 로 변경 범위 확인

패턴을 발견하면 고치는 데서 끝내지 말고 **CLAUDE.md 규칙**이나 **테스트**로 재발을 막습니다

모듈 4

AI에게 리뷰 맡기기 — 다른 눈으로 교차 검증

방법	어떻게	장점	단점	언제 쓰나
서브 에이전트	"서브 에이전트로 리뷰해"	현재 세션 유지 · 요약만 받아 컨텍스트 절약	리뷰 과정이 안 보여 되묻기 어려움	기본값 — 평소 모든 리뷰
새 세션	`/clear` 후 리뷰 요청	리뷰 과정을 보며 되물을 수 있음 · 선입견 완전 차단	작업 맥락이 사라져 리뷰 후 다시 설명	지적을 깊이 파고들 때
다른 모델	`/model` 로 바꿔 리뷰	모델별 사각지대가 달라 반복 누락을 잡음	전환·복귀 조작, 비용↑ · 새 세션과 함께 써야 효과	같은 실수가 반복될 때

PROMPT · 새 세션 또는 서브 에이전트

```
› @src/routes/tasks.ts 를 @spec.md · @CLAUDE.md 기준으로 리뷰해 줘.
  - 계약(상태 코드)·검증·오류 처리 위주로
  - 문제마다 파일·줄과 재현 방법(입력 · 기대 응답)을 함께
  - 고치지 말고 보고만 해 줘
```

그래도 판단은 사람이 - 지적이 맞는지 · 고칠 가치가 있는지는 **사람이 결정**

모듈 4

AI 코딩 보안 — 권한 규칙과 기본 주의사항

```
// .claude/settings.json (팀 공유)

"permissions": {
  "allow": ["Bash(npm test:*)", "Bash(npx prisma generate)",
            "Bash(git status:*)"],
  "ask":   ["Bash(git push:*)", "Bash(npm install:*)",
            "Bash(npx prisma migrate reset:*)"],
  "deny":  ["Read(./.env)", "Read(./.env.*)",
            "Edit(./.env*)", "Bash(rm -rf:*)"]
}
```

→ 규칙은 모든 권한 모드보다 우선

- 적용 순서 **deny → ask → allow → 현재 모드**
- allow는 승인 시 "don't ask again"으로 **쓰면서 쌓임**
- deny·ask는 **사람이 직접 설계** — 비밀 파일·파괴적 명령

비밀정보는 컨텍스트 밖에
.env 읽기 deny, 프롬프트에 키 금지
→ 키를 대화에 붙여 넣었다면 노출된 것 — 바로 폐기하고 새로 발급

파괴적 명령은 사람이 승인
DB 초기화·rm·force push
→ 승인 창에서 대상 경로·DB 이름을 읽고 누르기 — 습관적 '예' 금지

생성 코드의 보안 검토
입력 검증·인증·정보 노출
→ 새 세션에 보안 관점 리뷰 요청 — 오류 응답의 내부 정보 노출 확인

외부 콘텐츠의 지시 경계
웹·문서 속 지시문 (인젝션)
→ 읽어 온 내용이 명령 실행을 요구하면 따르지 말고 멈춰서 확인

도구가 막아 주는 경우도 있지만, 기본은 **권한 규칙 + 사람의 승인**입니다

05

프로젝트 문서화와 운영 설정

- CLAUDE.md 작성 — 프로젝트 규칙 정의
- README와 API 문서 자동 생성
- .gitignore, 환경변수 등 운영 설정
- 문서가 AI 협업 품질에 미치는 영향 · 과정 마무리

모듈 5

CLAUDE.md — AI 팀원에게 건네는 온보딩 문서

규칙·명령어·컨벤션을 한 번 정리해 두면 **새 세션마다 자동으로 로드** — 매번 다시 설명할 필요가 없습니다

위치	경로	적용 범위
사용자 전역	~/.claude/CLAUDE.md	내 모든 프로젝트
프로젝트 루트	./CLAUDE.md	팀 공유 — Git에 커밋
개인 로컬	./CLAUDE.local.md	나만 — .gitignore 필수
하위 폴더	src/CLAUDE.md	해당 폴더 작업 시

넣을 것

- 기술 스택
- 실행·테스트 명령
- 폴더 역할
- 코딩 규칙
- Git 규칙

뺄 것

- 비밀번호·API 키
- README 중복
- 린터가 강제하는 규칙
- 낡은 지침

1. **/init** — 코드 분석해 기존 CLAUDE.md 보완 제안
2. **검토** — 틀린 명령·과한 설명 정리
3. **규칙 추가** — 수업 중 반복된 실수를 규칙으로
4. **커밋** — 팀과 공유 — 모든 세션이 같은 규칙

CLAUDE.md는 **짧게** — 상세 문서는 `@spec.md`, `@README.md` 로 참조해 중복을 줄입니다

모듈 5 · 실습

문서 자동 생성 — README · 설계서 · 설명서

① README — Claude Code 로 (실습)

PROMPT · README

```
› 이 프로젝트의 README.md 를 작성해 줘.
  - 소개, 기술 스택, 설치·실행 순서, 환경변수 표
  - API 엔드포인트 표와 curl 예시
  - 테스트 실행 방법, 폴더 구조
  - 브라우저 화면 접속 방법 (http://localhost:3000)
  추측하지 말고 package.json 과 src 코드를
  근거로 작성하고, 실제로 명령을 실행해 확인해 줘.
```

② 설계서·설명서 — Claude Desktop 으로 작성

① **폴더 연결** — "프로젝트 또는 폴더"에서 실습 폴더 선택

② **프롬프트**

› spec.md 와 src 코드를 근거로 설계서를 docx 로 만들어 줘. 구조도·데이터 모델·API·테스트 전략을 포함해.

› public/ 화면을 기준으로 사용 설명서를 docx 로 만들어 줘.

③ **검토** — 코드와 대조해 없는 기능이 적혀 있지 않은지 확인

문서	독자	만드는 도구 → 형식
README	개발자·동료	Claude Code → README.md (실습)
설계서	기획자·리뷰어·운영	Claude Desktop → docx (시연) — 구조도·데이터 모델·API·테스트 전략
사용 설명서	최종 사용자	Claude Desktop → docx·pdf — public/ 화면 기준

어느 문서든 **코드가 근거** — 사람이 대조해 검증 · 코드가 바뀌면 문서도 갱신

모듈 5

.gitignore와 환경변수 — 운영 설정의 기본

```
# .env  (커밋 X — 실제 값)
DATABASE_URL="file:./dev.db"
PORT=3000
```

```
# .env.example  (커밋 O — 양식)
DATABASE_URL="file:./dev.db"
PORT=3000
```

새 팀원은 `cp .env.example .env` 후 값만 채움

.gitignore

```
node_modules/

# 환경변수 — .env.example 만 커밋
.env
.env.*
!.env.example

# SQLite DB · Prisma 생성 코드
*.db
src/generated/

# Claude Code 개인 설정
.claude/settings.local.json
CLAUDE.local.md
```

공유 vs 개인

- **공유**: CLAUDE.md, .claude/settings.json, .env.example
- **개인**: CLAUDE.local.md, settings.local.json, .env
- AI에게도 같은 원칙 — `.env` 읽기는 deny

커밋 전 `git status` 로 **.env·DB 파일이 빠졌는지** 확인 — AI가 만든 파일도 예외 없이

모듈 5

문서가 AI 협업 품질을 결정한다



	문서가 없을 때	문서가 있을 때
기술 선택	세션마다 다름	고정
테스트	자주 생략	자동 실행·통과 확인
오류 형식	엔드포인트마다 제각각	일관됨
새 세션·새 팀원	처음부터 다시 설명	즉시 파악

실제 사례 — Discourse

- AI 규칙을 **AI-AGENTS.md 하나**에 정리
- CLAUDE.md 등은 이 파일을 가리키는 링크
- 여러 AI 도구가 **같은 규칙**을 공유 (CLAUDE.md · AGENTS.md · GEMINI.md)

원문 보기
github.com/discourse/discourse
→ AI-AGENTS.md

모듈 5

과정 정리 — 에이전틱 엔지니어링 체크리스트

1. **스펙과 계획이 먼저**
   코드 전에 spec.md, 작업은 Plan 모드로 시작

2. **컨텍스트가 품질을 결정**
   필요한 파일만 @, 외부 문서는 XML 태그로

3. **작업은 작게 나눈다**
   한 작업 = 한 세션 = 테스트로 끝나는 단위

4. **검증 수단을 준다**
   테스트 피드백 루프, 통과가 완료 기준

5. **권한은 작업에 맞게**
   모드는 작업마다 확인, deny·ask는 직접 설계

6. **사람의 판단 지점 두 곳**
   계획을 승인하는 순간, 결과를 확인하는 순간

7. **반복되면 문서로**
   같은 지적이 두 번 나오면 CLAUDE.md에

8. **공식 문서를 확인**
   최신 라이브러리는 Context7·공식 문서로

과정 마무리

AI를 팀원으로 쓰는 엔지니어

깊이 이해하고, 더 나은 지시를 내리고, 결과를 검증합니다